

Unidad 1. Arquitecturas Web

1. Introducción

Las arquitecturas web constituyen el fundamento sobre el cual se construyen las aplicaciones y servicios que utilizamos diariamente en Internet. Comprender estos modelos arquitectónicos es esencial para cualquier profesional del desarrollo y despliegue de aplicaciones web, ya que determinan aspectos cruciales como el rendimiento, la escalabilidad, la seguridad y el mantenimiento de los sistemas.

Una arquitectura web define la estructura y organización de los componentes que conforman una aplicación, estableciendo cómo se comunican entre sí, cómo se distribuyen las responsabilidades y cómo se gestionan los recursos. La elección de una arquitectura adecuada puede marcar la diferencia entre un sistema robusto y escalable, y uno que presenta problemas constantes de rendimiento y mantenibilidad.

2. Arquitectura Cliente-Servidor

¿Qué es y por qué es importante?

La arquitectura cliente-servidor es como el sistema nervioso de Internet. Es el modelo que hace posible que podamos navegar por páginas web, ver vídeos en streaming, revisar nuestro correo electrónico o comprar online. Podemos pensar en ella como una conversación organizada entre dos partes: una que pide cosas (el cliente) y otra que las proporciona (el servidor).

Este modelo es tan exitoso que prácticamente todas las aplicaciones web que usamos a diario funcionan bajo este principio: desde redes sociales hasta plataformas de trabajo colaborativo.

Los dos protagonistas: Cliente y Servidor

El Cliente: Tu ventana al mundo digital

El cliente es la parte de la aplicación con la que interactúas directamente. En la mayoría de los casos, es tu navegador web (Chrome, Firefox, Safari, Edge, etc.). Piensa en el cliente como un mensajero personal: su trabajo es tomar tus peticiones, enviarlas al lugar correcto y mostrarte las respuestas de forma comprensible.

¿Qué hace exactamente el cliente?

- Presenta la interfaz visual que ves en tu pantalla
- Captura tus acciones (clics, escritura, desplazamientos)
- Envía solicitudes al servidor cuando necesitas información
- Recibe y muestra los resultados de manera atractiva y funcional
- Se ejecuta directamente en tu dispositivo (ordenador, móvil, tablet)

El Servidor: El cerebro detrás de la operación

El servidor es un ordenador especializado que funciona las 24 horas del día esperando peticiones. No tiene pantalla ni teclado visible; su única misión es procesar solicitudes y

enviar respuestas. Es como la cocina de un restaurante: tú no la ves, pero ahí es donde se prepara todo lo que necesitas.

¿Qué hace el servidor?

- Espera y recibe solicitudes de múltiples clientes simultáneamente
- Ejecuta la lógica de negocio (cálculos, validaciones, reglas)
- Consulta y actualiza bases de datos
- Procesa información y genera respuestas personalizadas
- Gestiona la seguridad y los permisos de acceso
- Almacena los datos de forma permanente y segura

¿Cómo se comunican?

La comunicación entre cliente y servidor sigue un protocolo establecido llamado HTTP (Protocolo de Transferencia de Hipertexto). Es como un idioma común que ambos entienden perfectamente.

El ciclo de una petición: un ejemplo práctico

Imaginemos que entras en una red social y quieres ver tu feed de noticias:

1. **Tú (usuario) actúas:** Abres la aplicación o recargas la página
2. **El cliente envía la petición:** Tu navegador envía un mensaje al servidor diciendo "Dame las últimas publicaciones del usuario123"
3. **El servidor procesa:**
 - Verifica tu identidad (¿estás conectado?)
 - Consulta la base de datos buscando publicaciones recientes
 - Aplica filtros según tus preferencias
 - Ordena las publicaciones por relevancia o fecha
4. **El servidor responde:** Envía los datos de las publicaciones al cliente
5. **El cliente muestra el resultado:** Tu navegador organiza y presenta esas publicaciones de forma visual y atractiva en tu pantalla

Todo este proceso suele durar apenas fracciones de segundo.

2.1.Ventajas de este modelo

Separación clara de responsabilidades

Cada componente tiene su función específica. El cliente se preocupa de la presentación y la interacción con el usuario, mientras el servidor maneja la lógica compleja y los datos. Es como tener especialistas para cada tarea.

Centralización de datos y lógica

Toda la información importante y las reglas de negocio están en el servidor. Esto significa que:

- Los datos están seguros y respaldados en un lugar controlado
- Las actualizaciones se hacen una sola vez en el servidor, no en miles de dispositivos
- Todos los usuarios trabajan con la misma versión de la aplicación

Escalabilidad

Cuando más personas empiezan a usar tu aplicación, puedes añadir más servidores para distribuir la carga de trabajo, sin que los usuarios noten ningún cambio.

Mantenimiento independiente

Puedes actualizar el diseño del cliente sin tocar el servidor, o mejorar la lógica del servidor sin cambiar la interfaz. Son piezas independientes que funcionan juntas.

Acceso multiplataforma

El mismo servidor puede atender a clientes muy diferentes: navegadores web, aplicaciones móviles, smartwatches, asistentes de voz, etc. Todos hablan el mismo idioma (HTTP) con el servidor.

Un ejemplo cotidiano: Comprar un libro online

Para entenderlo mejor, veamos qué sucede cuando compras un libro en una tienda online:

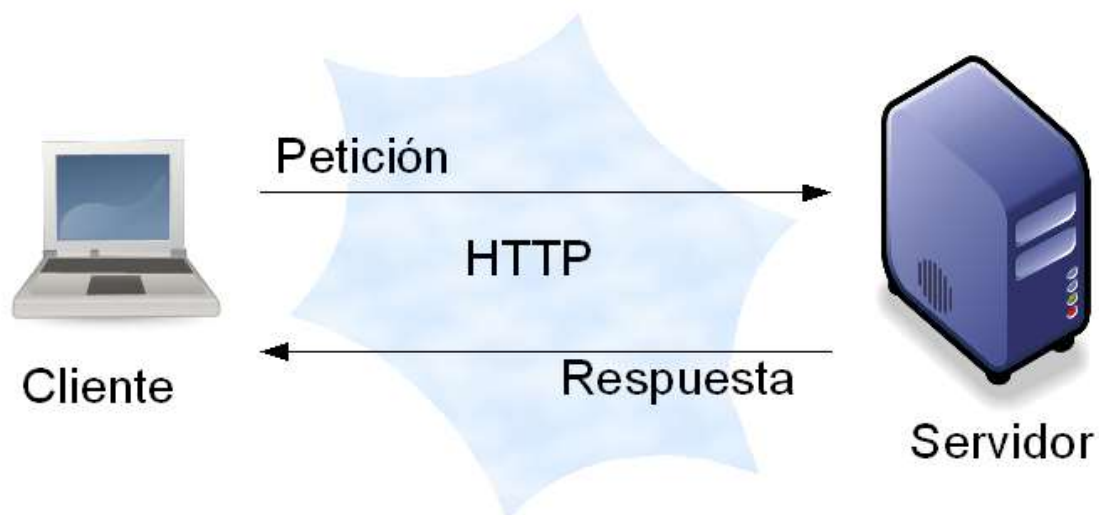
1. **Buscas el libro** → El cliente envía tu búsqueda al servidor
2. **Ves resultados** → El servidor consulta su catálogo y devuelve coincidencias
3. **Añades al carrito** → El cliente informa al servidor, que guarda temporalmente tu selección
4. **Procedes al pago** → El cliente muestra un formulario; tú introduces tus datos
5. **Confirmas la compra** → El servidor verifica el pago, actualiza el inventario, genera un pedido y te envía una confirmación
6. **Recibes confirmación** → El cliente muestra el mensaje de éxito y los detalles de tu pedido

En cada paso, cliente y servidor colaboran: uno presenta y recoge información, el otro procesa y toma decisiones.

2.2. Consideraciones modernas

Aunque el modelo básico sigue siendo el mismo, las aplicaciones modernas han evolucionado para ser más sofisticadas:

- **Cientes más inteligentes:** Los navegadores actuales pueden realizar operaciones complejas localmente, reduciendo la necesidad de consultar constantemente al servidor
- **Comunicación en tiempo real:** Tecnologías como WebSockets permiten que servidor y cliente mantengan una conexión abierta para actualizaciones instantáneas (chats, notificaciones)
- **Múltiples servidores:** Las aplicaciones grandes no usan un solo servidor, sino redes de servidores especializados trabajando en conjunto
- **Cachés intermedias:** Se almacenan copias temporales de datos frecuentes para acelerar las respuestas



3. Arquitectura de Tres Capas

¿Por qué tres capas y no dos?

Imagina que estás construyendo una casa. Podrías simplemente levantar paredes y poner un tejado (dos capas: estructura y cubierta). Pero los arquitectos saben que es mejor añadir una capa intermedia: cimientos, estructura habitable y tejado. Cada una tiene su función específica y pueden mejorarse o repararse independientemente.

La arquitectura de tres capas aplica este mismo principio al desarrollo de software. Es una evolución natural del modelo cliente-servidor básico que vimos anteriormente, pero con una organización más sofisticada y profesional. En lugar de tener simplemente "cliente" y "servidor", dividimos el trabajo en tres niveles especializados, cada uno con responsabilidades muy específicas.

Las tres capas: una orquesta bien coordinada

Piensa en las tres capas como los departamentos de una empresa bien organizada:

Capa de Presentación: El escaparate

¿Qué es? Es todo lo que el usuario ve y toca. La cara visible de tu aplicación. Es como el mostrador de atención al cliente en una tienda: el lugar donde interactúas directamente con el servicio.

¿Qué incluye?

- La interfaz gráfica: botones, formularios, menús, imágenes
- Las páginas web que ves en tu navegador
- Las pantallas de aplicaciones móviles
- Incluso aplicaciones de escritorio o interfaces de voz

¿Cuál es su trabajo?

- Mostrar la información de forma atractiva y comprensible
- Capturar lo que el usuario quiere hacer (clics, toques, comandos de voz)
- Validar que los datos introducidos tengan sentido básico (por ejemplo, que un email tenga formato de email)
- Enviar las solicitudes del usuario a la siguiente capa
- Recibir respuestas y presentarlas de forma visual

Ejemplo práctico: Cuando reservas un vuelo online, la capa de presentación te muestra el calendario para elegir fechas, el formulario para introducir tus datos y el botón para confirmar la reserva. Es todo lo visual y interactivo.

Capa de Lógica de Negocio: El cerebro de la operación

¿Qué es? Es el corazón inteligente de la aplicación. Aquí es donde viven todas las reglas, decisiones y procesos que hacen que tu aplicación funcione según las necesidades del

negocio. Es como el gerente de la tienda que toma decisiones según las políticas de la empresa.

¿Qué incluye?

- Las reglas del negocio (¿quién puede hacer qué?)
- Los cálculos y algoritmos (precios, descuentos, recomendaciones)
- La validación profunda de datos (¿este usuario tiene crédito suficiente?)
- La coordinación de operaciones complejas
- La aplicación de permisos y seguridad

¿Cuál es su trabajo?

- Recibir solicitudes de la capa de presentación
- Aplicar todas las reglas de negocio relevantes
- Realizar cálculos y procesamiento de información
- Decidir qué datos necesita consultar o guardar
- Coordinar con la capa de datos para obtener o almacenar información
- Devolver resultados procesados a la capa de presentación

Ejemplo práctico: Siguiendo con la reserva de vuelo, esta capa verifica que hay plazas disponibles, calcula el precio según la temporada y tu perfil de cliente, aplica descuentos si tienes un código promocional, comprueba que tu tarjeta tiene fondos suficientes y decide si la reserva puede confirmarse o no.

Capa de Acceso a Datos: El archivo y almacén

¿Qué es? Es el guardián de toda la información permanente. Esta capa se encarga exclusivamente de guardar y recuperar datos. Es como el almacén y el archivo de la empresa: todo está ordenado, catalogado y listo para ser consultado o actualizado.

¿Qué incluye?

- Las conexiones a bases de datos
- Las consultas para buscar información
- Las operaciones de guardado y actualización
- Los mecanismos de respaldo y recuperación
- La optimización del almacenamiento

¿Cuál es su trabajo?

- Recibir peticiones de la capa de lógica de negocio para obtener o guardar datos
- Traducir esas peticiones al lenguaje de la base de datos
- Ejecutar las consultas de forma eficiente

- Asegurar que los datos se guarden correctamente
- Mantener la integridad y consistencia de la información
- Devolver los datos solicitados en un formato utilizable

Ejemplo práctico: Esta capa busca en la base de datos los vuelos disponibles, recupera tu historial de reservas, guarda tu nueva reserva con todos los detalles, actualiza el número de plazas disponibles y registra la transacción de pago. Todo queda guardado de forma segura y permanente.

Cómo trabajan juntas: Un ejemplo completo

Veamos qué ocurre cuando compras un producto en una tienda online, paso a paso, capa por capa:

Paso 1: Buscas un producto

- **Presentación:** Escribes "zapatillas running" en el buscador y pulsas Enter
- **Lógica:** Recibe tu búsqueda, la procesa (corrige faltas, añade sinónimos) y pide a Datos que busque productos relacionados
- **Datos:** Consulta la base de datos y recupera todos los productos que coinciden
- **Lógica:** Ordena los resultados según relevancia, stock y valoraciones
- **Presentación:** Te muestra una lista atractiva con fotos, precios y valoraciones

Paso 2: Añades al carrito

- **Presentación:** Haces clic en "Añadir al carrito"
- **Lógica:** Verifica que el producto tiene stock disponible y que el precio es el actual
- **Datos:** Comprueba la disponibilidad en la base de datos
- **Lógica:** Si todo está bien, guarda el producto en tu carrito temporal
- **Presentación:** Te muestra una confirmación y actualiza el icono del carrito

Paso 3: Finalizas la compra

- **Presentación:** Introduces tu dirección y forma de pago, y confirmas
- **Lógica:** Valida que los datos sean correctos, calcula el total con impuestos y gastos de envío, verifica que hay stock, procesa el pago y genera el pedido
- **Datos:** Guarda el pedido completo, actualiza el inventario, registra el pago y almacena la dirección de envío
- **Lógica:** Confirma que todo se guardó correctamente y envía un email de confirmación
- **Presentación:** Te muestra la página de "Pedido confirmado" con todos los detalles

¿Por qué tres capas?

Podrías preguntarte: "¿No sería más simple tenerlo todo junto?". La respuesta es sí, sería más simple... pero solo al principio. A largo plazo, esta separación aporta beneficios enormes:

1. Mantenimiento más fácil

Cada capa puede evolucionar y mejorarse sin afectar a las demás. Es como tener habitaciones separadas en una casa: puedes redecorar el salón sin tocar la cocina.

Ejemplo real: Quieres cambiar el diseño visual de tu web porque se ha quedado anticuado. Con tres capas, solo modificas la capa de presentación. Toda la lógica de negocio y los datos siguen igual. No hay riesgo de romper funcionalidades críticas.

2. Reutilización de código

La misma lógica de negocio puede servir a múltiples interfaces diferentes.

Ejemplo real: Tienes una web de comercio electrónico y decides lanzar también una app móvil. La lógica de negocio (cálculos de precios, gestión de pedidos, validaciones) es exactamente la misma. Solo necesitas crear una nueva capa de presentación para móvil. Ahorras meses de desarrollo y evitas inconsistencias.

3. Equipos especializados

Diferentes desarrolladores pueden trabajar en diferentes capas simultáneamente sin estorbarse.

Ejemplo real: Tu equipo de diseñadores y desarrolladores frontend trabaja en mejorar la experiencia de usuario en la capa de presentación. Mientras tanto, otros desarrolladores optimizan los algoritmos de recomendación en la lógica de negocio. Y tu equipo de bases de datos mejora el rendimiento de las consultas en la capa de datos. Los tres equipos avanzan en paralelo.

4. Pruebas más efectivas

Cada capa puede probarse de forma aislada, detectando problemas más rápidamente.

Ejemplo real: Puedes probar que tu lógica de descuentos funciona correctamente sin necesidad de crear toda la interfaz visual. Simplemente alimentas la capa de lógica con datos de prueba y verificas que los cálculos son correctos. Esto acelera enormemente el proceso de control de calidad.

5. Cambios de tecnología sin drama

Puedes cambiar la tecnología de una capa sin afectar a las otras.

Ejemplo real: Tu base de datos actual se queda pequeña y decides migrar a una más potente. Como la capa de datos está aislada, puedes cambiarla completamente. La capa de lógica ni se entera: sigue pidiendo y recibiendo datos exactamente igual.

6. Escalabilidad independiente

Cada capa puede crecer a su propio ritmo según las necesidades.

Ejemplo real: Tu aplicación tiene mucho éxito y recibe millones de visitas. Notas que la capa de lógica de negocio es la que más se carga. Puedes añadir más servidores solo para esa capa, sin necesidad de multiplicar todo el sistema. Esto optimiza costos y recursos.

Reglas de oro para que funcione

Para que esta arquitectura sea efectiva, hay que seguir algunas reglas importantes:

Comunicación unidireccional

Las capas se comunican siempre en orden: Presentación → Lógica → Datos. La capa de presentación nunca debe hablar directamente con la de datos, saltándose la lógica. Sería como que un cliente entrara en el almacén sin pasar por el mostrador: caos asegurado.

Cada capa en su rol

La presentación no hace cálculos de negocio, la lógica no decide colores de botones, y los datos no validan reglas de negocio. Cada una se centra en lo suyo.

Interfaces claras

Las capas se comunican mediante "contratos" bien definidos. Cada capa sabe exactamente qué puede pedir a la siguiente y qué va a recibir, sin sorpresas.

Casos de uso ideales

Esta arquitectura es especialmente útil para:

- **Aplicaciones empresariales** con reglas de negocio complejas
- **Proyectos grandes** con múltiples equipos de desarrollo
- **Sistemas que necesitarán evolucionar** durante años
- **Aplicaciones multiplataforma** (web, móvil, escritorio)
- **Proyectos donde la lógica de negocio es el valor principal** del producto

Comparación visual: 2 vs 3 capas

Arquitectura de 2 capas (Cliente-Servidor):

Cliente (presenta + algo de lógica) ↔ Servidor (lógica + datos mezclados)

Arquitectura de 3 capas:

Presentación → Lógica de Negocio → Acceso a Datos

(muestra info) (toma decisiones) (guarda y recupera)

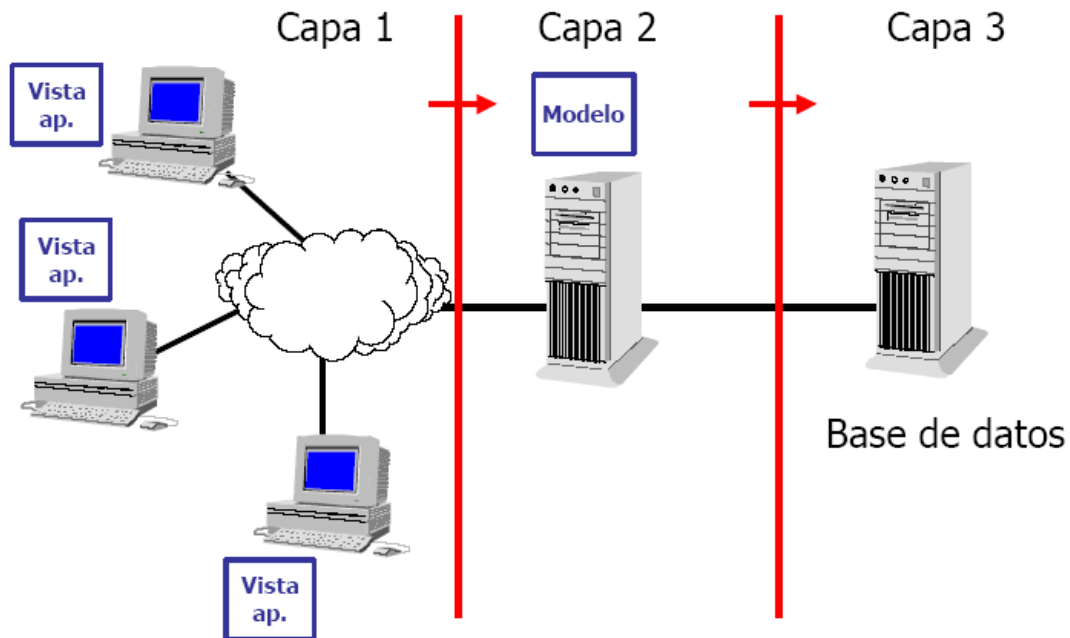
La diferencia es como pasar de una oficina de dos habitaciones a una de tres: más organizada, más profesional y más eficiente a largo plazo.

Conclusión

La arquitectura de tres capas no es solo una forma elegante de organizar código. Es una filosofía de diseño que reconoce que las aplicaciones complejas necesitan estructura, especialización y separación de responsabilidades. Al igual que un edificio bien diseñado

tiene cimientos, estructura y acabados diferenciados, una aplicación profesional beneficia enormemente de tener sus capas de presentación, lógica y datos claramente separadas.

Esta inversión inicial en organización se paga con creces en facilidad de mantenimiento, capacidad de evolución y calidad general del software. Es la diferencia entre construir algo que funciona hoy y construir algo que seguirá funcionando y mejorando durante años.



4.Arquitectura Orientada a Servicios (SOA)

Principios Fundamentales

La arquitectura orientada a servicios representa un paradigma que organiza las aplicaciones como conjuntos de servicios independientes y reutilizables. Cada servicio encapsula una funcionalidad específica del negocio y puede ser invocado de manera estandarizada a través de interfaces bien definidas.

Los servicios en SOA se caracterizan por ser autónomos, con bajo acoplamiento entre sí y alta cohesión interna. Esto significa que cada servicio puede ser desarrollado, desplegado y escalado independientemente, sin afectar a otros servicios del sistema.

Comunicación entre Servicios

La comunicación en SOA tradicionalmente se realiza mediante protocolos estándar como SOAP sobre HTTP, aunque también pueden utilizarse otros mecanismos. Los servicios publican contratos que definen sus interfaces, permitiendo que otros componentes del sistema los invoquen sin necesidad de conocer su implementación interna.

4.1.Arquitectura de Microservicios

Evolución desde SOA

Los microservicios representan una evolución y refinamiento de los principios SOA, llevando la modularización a un nivel más granular. En esta arquitectura, la aplicación se descompone en servicios muy pequeños, cada uno responsable de una única capacidad de negocio específica.

A diferencia de SOA, que a menudo implica servicios más grandes y complejos, los microservicios son deliberadamente pequeños y enfocados. Cada microservicio puede ser desarrollado por un equipo reducido, utilizar su propia tecnología y base de datos, y ser desplegado independientemente.

Características Distintivas

Los microservicios se comunican típicamente mediante protocolos ligeros como HTTP/REST o sistemas de mensajería asíncrona. Cada servicio es completamente independiente, con su propio ciclo de vida de desarrollo y despliegue. Esta independencia permite actualizaciones continuas y facilita la adopción de nuevas tecnologías de forma gradual.

La gestión de microservicios introduce complejidades adicionales, como la necesidad de sistemas de descubrimiento de servicios, balanceadores de carga, herramientas de monitorización distribuida y estrategias sofisticadas para manejar fallos en cascada.

Ventajas y Desafíos

Entre las ventajas destacan la escalabilidad granular (cada servicio puede escalarse independientemente según su demanda), la resiliencia mejorada (el fallo de un servicio no necesariamente afecta a todo el sistema) y la flexibilidad tecnológica.

Los desafíos incluyen la complejidad operacional aumentada, la necesidad de gestionar transacciones distribuidas, la dificultad para realizar pruebas de integración y la mayor demanda de recursos de infraestructura.

5. Arquitectura Serverless

Concepto y Fundamentos

La arquitectura serverless, o sin servidor, representa un modelo de ejecución en el que el proveedor de nube gestiona completamente la infraestructura subyacente. Los desarrolladores escriben funciones que se ejecutan en respuesta a eventos, sin preocuparse por la provisión, configuración o gestión de servidores.

En este modelo, el código se organiza en funciones pequeñas y específicas que se ejecutan bajo demanda. El proveedor de nube se encarga de toda la infraestructura, incluyendo el escalado automático, el balanceo de carga y la alta disponibilidad.

Modelo de Ejecución y Facturación

El modelo serverless introduce un cambio fundamental en la facturación: en lugar de pagar por servidores en funcionamiento continuo, se paga únicamente por el tiempo de

ejecución real de las funciones y los recursos consumidos durante esa ejecución. Esto puede resultar en ahorros significativos para aplicaciones con patrones de uso variables.

Las funciones serverless son típicamente stateless, lo que significa que no mantienen estado entre invocaciones. Cualquier estado necesario debe ser almacenado en servicios externos como bases de datos o sistemas de caché.

Casos de Uso Apropriados

Serverless es especialmente adecuado para aplicaciones con tráfico variable, procesamiento de eventos, tareas programadas, webhooks y APIs con patrones de uso impredecibles. Sin embargo, puede no ser óptimo para aplicaciones que requieren ejecución continua, tienen requisitos de latencia muy estrictos o necesitan mantener conexiones de larga duración.

6. Arquitectura de Aplicaciones de Página única (SPA)

Principios Operativos

Las aplicaciones de página única representan un enfoque donde la mayor parte de la lógica de la aplicación se ejecuta en el navegador del cliente. En lugar de cargar páginas completas del servidor, la SPA carga inicialmente el HTML, CSS y JavaScript necesarios, y posteriormente realiza actualizaciones dinámicas del contenido mediante llamadas API.

Este modelo transfiere significativamente más responsabilidad al cliente, que ahora se encarga del enrutamiento, la gestión del estado de la aplicación y la renderización de vistas. El servidor se convierte principalmente en un proveedor de datos a través de APIs REST o GraphQL.

Ventajas de Experiencia de Usuario

Las SPAs ofrecen experiencias de usuario más fluidas y responsivas, similares a las aplicaciones nativas. Las transiciones entre vistas son instantáneas, ya que no requieren recargar la página completa. Además, pueden funcionar parcialmente sin conexión mediante el uso de service workers y almacenamiento local.

Consideraciones Técnicas

El desarrollo de SPAs requiere frameworks como React, Vue o Angular, y presenta desafíos específicos como la gestión compleja del estado, el enrutamiento del lado del cliente, la optimización del tamaño del bundle JavaScript y consideraciones especiales para SEO, que tradicionalmente depende de contenido renderizado en el servidor.

7. Patrones de Despliegue

Despliegue Monolítico

En el despliegue monolítico, toda la aplicación se empaqueta y despliega como una única unidad. Esto simplifica el despliegue inicial y la gestión, pero puede complicar las actualizaciones y el escalado, ya que cualquier cambio requiere redespregar toda la aplicación.

Despliegue Distribuido

El despliegue distribuido divide la aplicación en múltiples componentes que pueden desplegarse independientemente en diferentes servidores o contenedores. Esto permite escalado selectivo y actualizaciones independientes, pero aumenta la complejidad operacional.

Contenedores y Orquestación

Los contenedores, mediante tecnologías como Docker, proporcionan un método estandarizado para empaquetar aplicaciones con todas sus dependencias. Los sistemas de orquestación como Kubernetes gestionan el despliegue, escalado y operación de contenedores en clústeres, facilitando la gestión de aplicaciones distribuidas complejas.

Consideraciones de Seguridad

La seguridad no es algo que añades al final, como un candado en la puerta. Debe estar presente en cada capa, en cada decisión de diseño, desde el primer día. Es como construir una casa: no puedes pensar en cerraduras y alarmas solo cuando ya está terminada; la seguridad debe estar integrada en los cimientos, las paredes y el tejado.

En una arquitectura web, la seguridad debe ser transversal: atraviesa todas las capas y todos los componentes. Ninguna capa puede confiar ciegamente en las demás. Es el principio de "confía, pero verifica... mejor aún, no confíes tanto".

Autenticación y Autorización: ¿Quién eres y qué puedes hacer?

Autenticación: Verificando identidades

La autenticación es el proceso de confirmar que alguien es quien dice ser. Es como mostrar tu DNI en un control de seguridad.

Métodos comunes:

- Contraseñas robustas: Combinación de caracteres, números y símbolos, almacenadas siempre cifradas (nunca en texto plano)
- Autenticación de dos factores (2FA): Además de la contraseña, un código temporal en tu móvil o email
- Biometría: Huella dactilar, reconocimiento facial

- Tokens de sesión: Una "credencial temporal" que el servidor te da después de iniciar sesión
- OAuth/Single Sign-On: Iniciar sesión con Google, Facebook, etc.

Ejemplo práctico: Cuando accedes a tu banco online, introduces tu usuario y contraseña (algo que sabes), luego te envían un código SMS (algo que tienes). Solo combinando ambos factores puedes entrar.

Autorización: Controlando permisos

Una vez que sabemos quién eres, necesitamos saber qué puedes hacer. Es como tener una tarjeta de acceso en una empresa: puede que te deje entrar al edificio, pero no a todas las habitaciones.

Niveles de autorización típicos:

- Usuario básico: Puede ver y editar solo su propio contenido
- Moderador: Puede ver y moderar contenido de otros usuarios
- Administrador: Tiene acceso a configuración y funciones críticas
- Superadministrador: Acceso total al sistema

Ejemplo práctico: En una plataforma de educación online, un estudiante puede ver cursos y hacer exámenes, un profesor puede crear contenido y calificar, pero solo un administrador puede gestionar usuarios y ver estadísticas globales.

Protección contra Ataques Comunes: Los enemigos conocidos

1. Inyección SQL: El ataque de los datos maliciosos

¿Qué es? Es cuando un atacante introduce código SQL malicioso en campos de entrada, intentando manipular tu base de datos.

Ejemplo de ataque: Imagina un formulario de login. El atacante, en lugar de introducir su nombre de usuario, escribe:

`admin' OR '1'='1`

Si tu código no está protegido, esto podría engañar a la base de datos para dar acceso sin contraseña.

Cómo protegerse:

- Usar consultas preparadas o consultas parametrizadas: nunca concatenar directamente lo que el usuario escribe
- Validar y sanitizar todas las entradas
- Aplicar el principio de menor privilegio en la base de datos (el usuario de la aplicación no debería poder borrar tablas)

2. Cross-Site Scripting (XSS): El ataque del código inyectado

¿Qué es? Un atacante consigue inyectar código JavaScript malicioso en tu página web, que luego se ejecuta en el navegador de otros usuarios.

Ejemplo de ataque: Un usuario malintencionado escribe en un comentario de un blog:

```
<script>robarCookies(</script>
```

Si tu web muestra ese comentario sin procesarlo, el código se ejecutará en el navegador de todos los que lean ese comentario, potencialmente robando sus sesiones.

Cómo protegerse:

- Escapar todo contenido generado por usuarios antes de mostrarlo en HTML
- Usar librerías modernas que escapan automáticamente (como React, Vue, Angular)
- Implementar Content Security Policy (CSP): le dice al navegador qué código JavaScript puede ejecutar
- Nunca usar eval() o funciones similares con datos de usuario

3. Cross-Site Request Forgery (CSRF): El ataque del formulario falso

¿Qué es? Engañar a tu navegador para que haga acciones en un sitio donde tienes sesión abierta, sin que tú lo sepas.

Ejemplo de ataque: Tienes tu sesión bancaria abierta. Visitas una web maliciosa que contiene un formulario invisible que se envía automáticamente a tu banco, haciendo una transferencia. Como tu navegador tiene la sesión activa, el banco piensa que realmente quieres hacerla.

Cómo protegerse:

- Tokens CSRF: cada formulario incluye un código secreto único que el atacante no puede conocer
- Verificar el header Referer (de dónde viene la petición)
- Usar cookies con el flag SameSite
- Para acciones críticas, requerir confirmación adicional

Cifrado de Datos: Secretos bien guardados

En tránsito: HTTPS siempre

Toda comunicación entre cliente y servidor debe ir cifrada mediante HTTPS (TLS/SSL). Es como enviar una carta en un sobre sellado en lugar de una postal que cualquiera puede leer.

Qué protege:

- Nombres de usuario y contraseñas

- Datos personales
- Información de pago
- Cualquier comunicación entre tu navegador y el servidor

Cómo implementarlo:

- Obtener un certificado SSL (gratuitos con Let's Encrypt)
- Configurar el servidor para forzar HTTPS
- Activar HSTS para que el navegador siempre use HTTPS

En reposo: Datos protegidos incluso si roban el disco

Los datos sensibles almacenados en bases de datos también deben estar cifrados. Si alguien roba físicamente el servidor o accede al disco duro, no podrá leer la información.

Qué cifrar:

- Siempre: Contraseñas (con algoritmos como bcrypt o Argon2, nunca cifrado simple)
- Datos sensibles: Números de tarjetas, datos médicos, documentos de identidad
- Según regulación: Datos personales si tu jurisdicción lo requiere (GDPR, etc.)

Ejemplo práctico: Una contraseña como "MiContraseña123" no se guarda tal cual en la base de datos. Se guarda como algo así:

\$2y\$10\$N9qo8uLOickgx2ZMRZoMyeljZAgcf17p92ldGxad68LJZdL17lhWy

Es imposible convertir esto de vuelta a la contraseña original. Cuando el usuario inicia sesión, se cifra lo que escribió y se compara con esto.

Principios Fundamentales de Seguridad

1. Principio de Menor Privilegio

Cada componente, usuario o servicio debe tener únicamente los permisos mínimos necesarios para hacer su trabajo, nada más.

Ejemplos prácticos:

- El código que muestra productos en tu web no necesita permisos para borrar usuarios
- Un empleado de atención al cliente puede ver pedidos pero no modificar precios
- La conexión a la base de datos desde la aplicación web solo puede hacer SELECT e INSERT en ciertas tablas, no DROP TABLE

Por qué es importante: Si un atacante compromete un componente con privilegios limitados, el daño que puede hacer también es limitado.

2. Defensa en Profundidad

No confíes en una única línea de defensa. Implementa múltiples capas de seguridad, de modo que si una falla, las demás siguen protegiendo.

Capas de defensa típicas:

1. Firewall: Bloquea tráfico sospechoso antes de que llegue al servidor
2. WAF (Web Application Firewall): Filtra peticiones HTTP maliciosas
3. Validación en cliente: Primera comprobación rápida (pero nunca confiar solo en ella)
4. Validación en servidor: Verificación real de los datos
5. Consultas parametrizadas: Protección contra inyección SQL
6. Cifrado: Incluso si alguien roba los datos, no puede leerlos
7. Logs y monitorización: Detecta intentos de intrusión en tiempo real
8. Backups: Si todo falla, puedes recuperar

Analogía: Es como proteger tu casa con cerradura en la puerta, alarma, cámara de seguridad y caja fuerte para los objetos de valor. Si un ladrón supera la cerradura, aún tiene que lidiar con la alarma.

3. No Confiar en el Cliente

Nunca asumas que el cliente (navegador, app móvil) sigue las reglas. Un atacante puede modificar cualquier cosa que ocurra en el lado del cliente.

Errores comunes:

- ✗ Validar solo en JavaScript del navegador (puede desactivarse)
- ✗ Ocultar botones de administrador con CSS (pueden hacerse visibles)
- ✗ Confiar en parámetros de URL o cookies sin verificar
- ✗ Asumir que los datos llegan en el formato esperado

Regla de oro: Todo lo que viene del cliente debe tratarse como potencialmente malicioso hasta que se verifique en el servidor.

Mantente Actualizado: La Seguridad es un Proceso

La seguridad no es algo que implementas una vez y olvidas. Nuevas vulnerabilidades se descubren constantemente.

Buenas prácticas:

- Actualiza dependencias regularmente: Librerías y frameworks publican parches de seguridad
- Auditorías de seguridad periódicas: Revisa tu código y configuración

- Pruebas de penetración: Contrata expertos para intentar hackear tu sistema
- Monitorización continua: Detecta comportamientos anómalos
- Plan de respuesta a incidentes: Sabe qué hacer si te hackean

Checklist de Seguridad Básica

Antes de lanzar cualquier aplicación web, verifica:

- HTTPS activado y forzado en todas las páginas
- Autenticación robusta implementada
- Sistema de autorización por roles funcionando
- Todas las entradas de usuario validadas en el servidor
- Protección contra inyección SQL (consultas parametrizadas)
- Protección contra XSS (escapado de contenido)
- Tokens CSRF en todos los formularios
- Contraseñas cifradas con algoritmos modernos
- Datos sensibles cifrados en la base de datos
- Principio de menor privilegio aplicado
- Logs de seguridad activados
- Plan de backups y recuperación probado
- Dependencias actualizadas a versiones sin vulnerabilidades conocidas

En resumen

La elección de una arquitectura web apropiada depende de múltiples factores: los requisitos funcionales y no funcionales del proyecto, las restricciones de recursos, la experiencia del equipo de desarrollo y las expectativas de crecimiento futuro. No existe una arquitectura universalmente superior; cada una ofrece compromisos diferentes entre complejidad, rendimiento, escalabilidad y facilidad de mantenimiento.

El profesional del despliegue de aplicaciones web debe comprender estas diferentes arquitecturas, sus ventajas, limitaciones y casos de uso apropiados, para poder tomar decisiones informadas que resulten en sistemas robustos, eficientes y mantenibles a largo plazo.

Bibliografía y referencias

Caballero, F. J., & Vázquez, R. (2022). *Administración de redes y servicios de red*. Madrid: Paraninfo.

López, M., & García, A. (2021). *Configuración y administración de servicios de red en sistemas operativos de servidor*. RA-MA Editorial.

Ministerio de Educación y Formación Profesional. (2020). *Guía de aprendizaje del módulo: Servicios de red e Internet (ASIR)*. Madrid: Gobierno de España.

Redondo, J. M., & Salas, P. (2019). *Infraestructuras de red y servicios en entornos corporativos*. Editorial Marcombo.